

ER Model & Database Design

1. The Database Design Process

Mini-world (the real-world enterprise/domain being modeled)

1. Requirements Collection & Analysis

→ talk to users, determine data requirements (what entities/attributes/relationships must exist) and functional requirements (what operations the system must support)

2. Conceptual Design

→ model the data using a high-level, DBMS-independent model – the ER Model

3. Logical Design (Data Model Mapping)

→ convert the conceptual schema into the logical schema of a specific chosen DBMS type (e.g., ER → Relational Model)

4. Physical Design

→ decide file structures, indexes, and access paths for efficient storage

5. Application Program / Transaction Implementation

Why go through the ER model first, rather than designing tables directly? The ER model is deliberately **DBMS-independent** and close to how non-technical stakeholders naturally think about their domain (entities, their properties, and how they relate) — this makes it easy to validate the design *with the actual users* before committing to any specific technology's constraints. Only after the conceptual design is validated does logical design translate it into a specific data model (relational, in most cases).

2. Core ER Concepts

Entity & Entity Set

- **Entity** — a real-world "thing" or object, distinguishable from every other object (e.g., a specific employee, "John, employee #21").
- **Entity type** — describes the schema/structure shared by a collection of similar entities (e.g., "Employee" — defined by attributes `Name`, `Age`, `Salary`).
- **Entity set** — the actual collection of entities of a given entity type at a point in time (also called the **extension** of the entity type).

Entity type: Employee (Name, Age, Salary)

Entity set: { (John, 55, 80000), (Mary, 42, 95000), ... } ← actual entities right now

Attributes and their types

Type	Meaning	Example
Simple (atomic)	Cannot be divided into smaller meaningful parts	Age , Gender
Composite	Can be divided into smaller sub-parts, each independently meaningful	Address → Street, City, State, Zip
Single-valued	Holds exactly one value per entity	Age
Multi-valued	Can hold multiple values for a single entity	PhoneNumbers (a person can have several)
Stored	Value is physically stored in the database	BirthDate
Derived	Computed from other stored attributes, not stored itself	Age (derived from BirthDate and current date)
Complex	Both composite <i>and</i> multi-valued, or arbitrarily nested	Multiple past addresses, each itself composite: {Address1(Street, City, ...), Address2(Street, City, ...)}
NULL	Special marker for "not applicable" or "unknown"	CollegeDegree for someone without one

```
      Name
     /  \
  FirstName  LastName ← composite attribute

Address (composite AND multi-valued = complex)
|
Address1{Street, City, State, Zip}, Address2{...}
```

Key attribute — an attribute whose value is unique for every entity in the entity set (e.g., `EmployeeID`).

Composite key — a key made up of two or more attributes *together* being unique, where no single one of them is unique alone (e.g., `RegistrationNumber` + `State` for a vehicle — neither alone is unique, but the pair is).

Rule: a composite key must be **minimal** — it should include only the attributes actually necessary for uniqueness.

Value set (domain) — the set of all possible legal values an attribute can take. Formally, an attribute `A` of entity set `E` with value set `V` is a function $A: E \rightarrow P(V)$ (mapping each entity to a *set* of possible values — a singleton set for single-valued attributes, a proper subset for multi-valued ones).

3. Relationships

- **Relationship type** — an association among entity types (e.g., `WORKS_FOR` between `Employee` and `Department`).
- **Relationship set** — the actual collection of relationship instances of that type at a point in time. Formally, $R \subseteq E_1 \times E_2 \times \dots \times E_n$.
- **Degree of a relationship type** — the number of entity types participating in it.

Degree	Name	Example
2	Binary	<code>Employee WORKS_FOR Department</code>
3	Ternary	<code>Supplier SUPPLIES Part TO Project</code>

Representing relationships as attributes (a special case)

A **1:1** or a relationship where one side can absorb the identifying attribute can sometimes be represented by simply adding an attribute — e.g., `WORKS_FOR` could be represented by adding a `DeptID` attribute directly to `Employee` , instead of maintaining a separate explicit relationship structure. **Constraint:** if both directions add such attributes, they must be exact inverses of each other.

Role names & recursive relationships

- **Role name** — specifies the role an entity plays within a specific relationship instance — essential when the **same entity type participates more than once** in one relationship (a **recursive relationship**).

Worked example — Supervision: an `Employee` participates twice in a `SUPERVISION` relationship — once as **Supervisor**, once as **Subordinate**. Without role names, there'd be no way to distinguish which occurrence means what.

4. Structural Constraints on Relationships

Cardinality Ratio (mapping cardinality)

Specifies the **maximum** number of relationship instances an entity can participate in.

Ratio	Meaning	Example
1:1	One entity of each type relates to at most one of the other	Employee MANAGES Department (one employee manages at most one dept; one dept has at most one manager)
1:N	One entity of the first type relates to many of the second, but each of the second relates to only one of the first	Department WORKS_FOR← Employee (one dept has many employees; each employee belongs to one dept)
N:1	The mirror image of 1:N	(Same relationship, viewed from the other side)
M:N	Entities on both sides can relate to many on the other side	Employee WORKS_ON Project (an employee can work on many projects; a project can have many employees)

Participation Constraint (minimum cardinality / existence dependency)

Specifies whether an entity's existence **depends on** participating in the relationship.

Type	Meaning	Example
Total participation	<i>Every</i> entity must participate in at least one relationship instance	Every Employee must work for some Department
Partial participation	Only <i>some</i> entities participate	Not every Employee manages a department

Structural constraints = cardinality ratio + participation constraint, together — these two, combined, fully describe the "shape" of allowed relationship configurations.

ER Diagram notation for these constraints

E1 —R— E2	(basic relationship, no constraint shown)
E1 ==R== E2	(double line = total participation of E2 in R)
E1 —R—(1,N)— E2	((min, max) notation directly specifies BOTH cardinality ratio (max) and participation (min) in one compact pair)

5. Weak Entities

A **weak entity type** has no key attribute of its own sufficient to uniquely identify its entities — it relies on a relationship to another entity type (its **owner**) for identification.

- **Owner entity** — the (strong) entity type that identifies the weak entity.
- **Partial key** — an attribute that, *combined with the owner's key*, uniquely identifies a weak entity (unique only *within* the scope of a specific owner, not globally).
- **Identifying relationship** — the relationship connecting a weak entity to its owner.
- **Identification rule:** a weak entity is uniquely identified by (partial key, owner entity) together.

Worked example: a **Dependent** entity (e.g., an employee's child, for insurance purposes) might have no globally unique ID of its own — `Dependent(Name, BirthDate)` — but `Name` is unique *for a given employee* (the owner). So a specific dependent is identified as `(EmployeeID, DependentName)`.

- **Weak entities have total participation** in their identifying relationship (they cannot exist without their owner).
- Multiple levels of weak entities are possible (a weak entity can itself own another weak entity).

6. ER Diagram Notation — Summary

Symbol	Meaning
Rectangle	Entity type
Double-outlined rectangle	Weak entity type
Diamond	Relationship type
Double-outlined diamond	Identifying relationship
Oval	Attribute
Underlined oval	Key attribute
Double-outlined oval	Multivalued attribute
Ovals connected in a tree	Composite attribute
Dashed oval	Derived attribute
Single line	Partial participation
Double line	Total participation

7. Migration of Relationship Attributes into Entity Types

A useful practical shortcut when translating an ER design (relevant to the upcoming Relational Model / mapping topic):

Relationship cardinality	Can the relationship's own attributes migrate into an entity type?
1:1	Can migrate to <i>either</i> entity type
1:N	Can only migrate to the entity on the N side (the "many" side)
M:N	Cannot migrate to either side — must remain as a separate structure

Why does 1:N only allow migration to the N side? On the N side, each entity participates in at most **one** instance of the relationship (from its own perspective, it has exactly one associated "1" side entity), so a relationship attribute maps cleanly to a single value per N-side entity — a normal single-valued attribute. On the "1" side, a single entity could be associated with *many* N-side entities, each potentially carrying a *different* value for that relationship attribute — so it can't collapse into one single-valued attribute on the "1" side without losing information.

8. Types of Data Models — where ER fits

Model	Core idea
Relational Model	Data organized in tables (relations) with rows (tuples) and columns (attributes); relationships established through keys
ER Model	Conceptual model — entities, attributes, relationships (the subject of this topic)
Object-based Model	ER Model + OOP ideas — entities have both attributes <i>and</i> methods
Semistructured Model	For data without a rigid schema (structure can vary record to record) — e.g., JSON/XML, used in NoSQL databases
Hierarchical Model	Tree structure — each child has exactly one parent
Network Model	Graph structure — a child can have multiple parents

ER is a *design-time conceptual* model; relational (and the others) are *implementation* models the conceptual design eventually gets mapped into.

Interview Questions With Answers

Q1. What's the difference between an entity, an entity type, and an entity set?

Answer: An entity is one specific real-world object (e.g., a particular employee). An entity type defines the shared structure/schema for a collection of similar entities (e.g., "Employee," defined by attributes like Name, Age, Salary). An entity set is the actual collection of entities of that type that currently exist in the database (the extension of the entity type) — the entity type is the blueprint; the entity set is the current data conforming to it.

Q2. Give an example of a complex attribute, and explain why it's called "complex" rather than just composite or multivalued.

Answer: A person's history of past addresses is a good example — it's multivalued (a person can have multiple past addresses) *and* each individual address is itself composite (Street, City, State, Zip). It's called complex specifically because it combines both properties simultaneously (or involves arbitrary nesting), rather than being purely one or the other — a plain composite attribute (like Name → FirstName/LastName) is single-valued, and a plain multivalued attribute (like PhoneNumbers) has atomic values; a complex attribute is a multivalued attribute whose individual values are themselves composite.

Q3. Why must a composite key be minimal?

Answer: A composite key's entire purpose is representing the smallest set of attributes that together guarantee uniqueness. If it included an attribute unnecessary for uniqueness (i.e., the remaining attributes would already be unique without it), it would technically still work as an identifier but would violate the definition of a *key* specifically — a superset of attributes that happens to be unique is a superkey, not a (minimal) key; including redundant attributes also makes the key artificially larger and more error-prone to use consistently.

Q4. Explain the difference between cardinality ratio and participation constraint, using the WORKS_FOR relationship between Employee and Department as an example.

Answer: Cardinality ratio specifies the maximum number of relationship instances an entity can participate in — for WORKS_FOR, this is 1:N (one department has many employees, but each employee belongs to only one department), describing the *shape* of the relationship. Participation constraint specifies the minimum — whether an entity's existence requires participating in the relationship at all — for WORKS_FOR, Employee typically has *total* participation (every employee

must work for some department) while Department might have *partial* participation (a newly created department might temporarily have zero employees). These are independent constraints answering different questions: "at most how many?" (cardinality) versus "is participation mandatory?" (participation).

Q5. What makes an entity type "weak," and why does a weak entity always have total participation in its identifying relationship?

Answer: A weak entity type has no attribute (or combination of its own attributes) sufficient to uniquely identify its entities on their own — it needs a partial key combined with a reference to its owner entity (via the identifying relationship) to be uniquely identified at all. It always has total participation in that identifying relationship because, by definition, a weak entity cannot even be meaningfully identified — let alone exist as a distinguishable record — without being linked to some owner entity; there's no valid state where a weak entity exists without that link, since its very identity depends on it.

Q6. Why do relationship attributes migrate cleanly to the "many" side of a 1:N relationship but not to the "one" side?

Answer: On the "many" (N) side, each entity is associated with exactly one entity on the "1" side — so any attribute describing that specific relationship instance has exactly one value per N-side entity, fitting naturally as a normal single-valued attribute of that entity. On the "1" side, a single entity can be associated with many N-side entities, each potentially needing a *different* value for that same relationship attribute — collapsing this into a single-valued attribute on the "1" side would either be wrong (only one value could be stored, losing the others) or require making it multivalued in a way that loses the clear pairing between each specific N-side entity and its corresponding attribute value.

Q7. Why can't relationship attributes be migrated into either entity type at all for an M:N relationship?

Answer: In an M:N relationship, entities on *both* sides can be associated with multiple entities on the other side, so a relationship attribute's value is tied specifically to a *particular pairing* of one entity from each side — not to either entity alone. Migrating it into either entity type would lose the information about which specific pairing that value belongs to (since either entity could be paired with several others, each potentially needing its own distinct value), so the relationship's own attributes must remain attached to the relationship itself (which, in the relational mapping, becomes its own table with foreign keys to both sides — a preview of the Relational Model topic).

Q8. Why is the ER model deliberately designed to be DBMS-independent, rather than modeling directly in terms of tables?

Answer: Conceptual design happens early, when the goal is to correctly capture what data and relationships the real-world domain actually requires — a step that should be validated with actual domain experts/users, most of whom don't think in terms of tables, foreign keys, or normalization. Keeping the ER model DBMS-independent (and conceptually closer to natural language descriptions

of entities and their relationships) makes this validation step accessible to non-technical stakeholders, and also means the same conceptual design could later be mapped to *any* target data model (relational, object-based, etc.) during logical design, rather than being prematurely locked into one specific technology's constraints.

Q9. Scenario: You're modeling a library system where a Book can have multiple Authors, and an Author can have written multiple Books. Additionally, each specific (Book, Author) pairing has a "royalty percentage" that varies per pairing. How would you model this, and why can't the royalty percentage just be an attribute of Author or Book?

Answer: This is an M:N relationship between Book and Author (a WRITTEN_BY or similar relationship type), and the royalty percentage must be an attribute of the *relationship itself*, not of either entity type. It can't be an attribute of Author, because a single author might have different royalty percentages for different books (one value wouldn't suffice per author). It can't be an attribute of Book either, if a book has multiple authors each with a potentially different royalty split. Since the value is specifically tied to one particular (Book, Author) pairing — exactly the situation where relationship attributes cannot migrate to either side of an M:N relationship — the royalty percentage must be modeled as an attribute of the WRITTEN_BY relationship type itself, which (in the relational mapping) becomes its own table containing foreign keys to both Book and Author, plus the royalty percentage column.